

Wnews Crew

Welcome to the Wnews Crew project, powered by [crewAI](#). This template is designed to help you set up a multi-agent AI system with ease, leveraging the powerful and flexible framework provided by crewAI. Our goal is to enable your agents to collaborate effectively on complex tasks, maximizing their collective intelligence and capabilities.

Installation

Ensure you have Python ≥ 3.10  .13 installed on your system. This project uses [UV](#) for dependency management and package handling, offering a seamless setup and execution experience.

First, if you haven't already, install uv:

```
pip install uv
```

Next, navigate to your project directory and install the dependencies:

(Optional) Lock the dependencies and install them by using the CLI command:

```
crewai install
```

Customizing

Add your [OPENAI_API_KEY](#) into the [.env](#) file

- Modify [src/wnews/config/agents.yaml](#) to define your agents
- Modify [src/wnews/config/tasks.yaml](#) to define your tasks
- Modify [src/wnews/crew.py](#) to add your own logic, tools and specific args
- Modify [src/wnews/main.py](#) to add custom inputs for your agents and tasks

Running the Project

To kickstart your crew of AI agents and begin task execution, run this from the root folder of your project:

```
$ crewai run
```

This command initializes the wnews Crew, assembling the agents and assigning them tasks as defined in your configuration.

This example, unmodified, will run the create a [report.md](#) file with the output of a research on LLMs in the root folder.

Understanding Your Crew

The wnews Crew is composed of multiple AI agents, each with unique roles, goals, and tools. These agents collaborate on a series of tasks, defined in `config/tasks.yaml`, leveraging their collective skills to achieve complex objectives. The `config/agents.yaml` file outlines the capabilities and configurations of each agent in your crew.

Support

For support, questions, or feedback regarding the Wnews Crew or crewAI.

- Visit our [documentation](#)
- Reach out to us through our [GitHub repository](#)
- [Join our Discord](#)
- [Chat with our docs](#)

Let's create wonders together with the power and simplicity of crewAI.

Wnews - AI Newsletter Generator

Wnews is a Python application built using the [CrewAI](#) framework. It automates the process of creating a newsletter focused on the impact of AI in business and recent AI technology news. It utilizes multiple AI agents collaborating to research relevant information, write insightful content, and edit the final newsletter.

Features

- Automated web research using SerperDevTool.
- AI-powered content generation and impact analysis.
- Collaborative workflow between specialized AI agents:
 - `ai_insights_curator`: Researches the latest AI news and developments.
 - `ai_content_writer`: Writes analysis based on the curated insights.
 - `newsletter_editor`: Reviews and edits the content into a final newsletter format.
- Generates intermediate insights (`insights.txt`) and a final newsletter in Markdown format (`r2talk_newsletter.md`).
- Supports training, replaying, and testing crew executions via command-line arguments.

Technology Stack

- Python 3.x
- CrewAI & CrewAI Tools
- SerperDevTool (for web search)

Setup and Installation

1. Prerequisites:

- Python 3.8 or higher installed.
- `pip` (Python package installer).

2. Clone the Repository:

```
git clone <your-repository-url> # Replace with your actual repo URL
cd wnews
```

3. Install Dependencies: It's recommended to use a virtual environment:

```
python -m venv venv
source venv/bin/activate # On Windows use `venv\Scripts\activate`
```

Install the required packages:

```
pip install -r requirements.txt
```

(Note: If *requirements.txt* does not exist, you'll need to create one based on the project's imports. Key dependencies include *crewai*, *crewai-tools*, *python-dotenv*, *google-search-results* (for *SerperDevTool*), and *pysbd*.)

4. Set up Environment Variables: This project uses SerperDevTool for web searches, which requires an API key.

- Sign up for an API key at Serper.dev.
- Create a file named *.env* in the root directory of the project (*wnews/*).
- Add your API key to the *.env* file:

```
SERPER_API_KEY='your_serper_api_key_here'
```

- CrewAI typically loads variables from a *.env* file automatically if *python-dotenv* is installed.

Usage

The main script *src/wnews/main.py* provides several commands:

1. Run the Newsletter Generation Crew: This command executes the default workflow: research, write, and edit.

```
python src/wnews/main.py run
```

This will use the topic defined in *main.py* (Impact of AI in Business, and recent news about AI technologies) and output *insights.txt* and *r2talk_newsletter.md* in the project's root directory.

2. **Train the Crew:** This command runs the crew multiple times for training purposes.

```
python src/wnews/main.py train <n_iterations> <output_filename.json>
```

- `<n_iterations>`: The number of training iterations.
- `<output_filename.json>`: The file to save training logs.
- Example: `python src/wnews/main.py train 5 training_log.json`

3. **Replay a Task:** Re-runs the crew starting from a specific task ID (useful for debugging).

```
python src/wnews/main.py replay <task_id>
```

- `<task_id>`: The ID of the task to replay from.

4. **Test the Crew:** Runs the crew multiple times for testing with a specific model.

```
python src/wnews/main.py test <n_iterations> <openai_model_name>
```

- `<n_iterations>`: The number of test iterations.
- `<openai_model_name>`: The OpenAI model to use (e.g., `gpt-4`, `gpt-3.5-turbo`).
- Example: `python src/wnews/main.py test 3 gpt-4`

Configuration

- **Agents:** Agent roles, goals, backstories, and tool assignments are defined in `config/agents.yaml`.
- **Tasks:** Task descriptions, expected outputs, and dependencies are defined in `config/tasks.yaml`.

You can modify these YAML files to customize the behavior, goals, and workflow of the AI crew.

Contributing

Contributions are welcome! Please feel free to submit pull requests or open issues to improve the project.

1. Fork the repository.
2. Create a new branch (`git checkout -b feature/your-feature-name`).
3. Make your changes.
4. Commit your changes (`git commit -m 'Add some feature'`).
5. Push to the branch (`git push origin feature/your-feature-name`).
6. Open a Pull Request.

License

[Specify Your License Here - e.g., MIT License]